

Explicit Provenance APIs

Document number: P3744R1.

Date: 2025-12-02.

Authors: Gonzalo Brito Gadeschi, David Chisnall.

Reply to: gonzalob *at* nvidia.com (<http://nvidia.com>), David.Chisnall *at* cheri-alliance.org (<http://cheri-alliance.org>) .

Audience: SG1, SG22, SG23.

Table of Contents

- Explicit Provenance APIs
- Changelog
- Synopsis
- Background
 - History and current state of C++
 - Uses of pointers as integers
 - C++ already permits most pointer-as-integer operations without round trips through `intptr_t`
- Core Proposal
 - The `ptraddr_t` type
 - Why not `size_t`?
 - Subtraction on `ptraddr_t`
 - Mandatory operations on `intptr_t`
 - Limitations with `intptr_t`
 - New strict-provenance APIs
- Examples
 - Concurrent queue
- Implementation Experience

- Optional extension
- C compatibility
- Note on naming
- Appendix: XOR linked list
- References

Changelog

- **R0**: initial revision.

Synopsis

Extends C++ with APIs that enable applications to explicitly work with provenance.

Applications that work with provenance only through these APIs benefit from a much simpler provenance/aliasing/object models, better compiler optimizations, and better hardware acceleration.

Applications that continue to work with provenance through pre-existing APIs are not impacted by the addition of these new APIs and they can continue to enjoy the complexity of an incomprehensible provenance models and its consequences (poor performance and poor hardware support).

Background

Pointer provenance is both a useful property for optimisation and a core requirement for any memory-safety guarantees.

Both spatial and temporal memory safety rely on the concept that a pointer to an object must be derived from an existing pointer to the object, for example taking the address of a value that has static or automatic storage duration or deriving a pointer from the return value from `operator::new` .

CHERI systems enforce this property in hardware as the foundation guarantee for all other memory-safety properties.

In a CHERI system, the compiler represents C++ pointers using a *CHERI Capability*.

This is a hardware-defined data type that contains a base, top, bounds, permissions, and an

address.

Every load, store, or jump instruction requires a CHERI capability as the base address and will perform bounds and permission checks.

At system boot, one or more registers contain a set of CHERI capabilities that authorise all accesses to memory.

An operating system with a traditional 50:50 address split may divide these into 'kernel' and 'userspace' halves by restricting the range.

Allocating new address space from the OS via `mmap`, `VirtualAlloc`, or similar calls will then return a new capability constructed by reducing the bounds on the userspace capability.

Similarly, a userspace `malloc` implementation will further subdivide this to hand out capabilities for individual allocations.

CHERI uses a non-addressable tag bit alongside each capability-sized word in memory to provide an attestation from the hardware that the accompanying value has followed a valid derivation chain.

If you have a valid CHERI capability in a register and move its address, reduce its bounds, reduce its permissions, or simply copy it to another register or store it to memory, then the result will also have the tag bit set.

If you overwrite part of the capability in memory with arbitrary data, the tag is cleared.

Tags therefore provide a hardware notion of pointer provenance: any pointer that is a valid pointer derived from another valid pointer will have its tag bit set.

The CHERI C/C++ model is designed to express operations that are not undefined behaviour in C or C++ and to be able to trap on ones that are.

Accessing an object via an out-of-bounds or dangling pointer, for example, are UB and CHERI platforms will trap on the first and may (depending on the software stack) trap on the second.

C++ is a systems language.

As a result, it permits operations on pointers that are excluded in application-programming languages but which are required to implement such languages.

This includes (but is not limited to):

- Implementing a memory allocator by subdividing static allocations or memory provided by a host platform.
- Storing data in bits that are known by the programmer (via some target-specific constraints) to be unused.

- Explicitly realigning storage within other objects.
- Using object addresses as identities for collections.

These all require some ability to interact with pointers at a lower level than simply as references to objects in some abstract space.

History and current state of C++

In BCPL, all values were words.

A word that was used as an address was a pointer.

ISO C standardised the distinction: pointers are pointers and integers are integers.

In most C implementations, this distinction was fairly loose and it was possible to cast between a pointer and an integer of the same size.

C++ inherited much of this behaviour.

C99 introduced `[u]intptr_t` types that were optional but, if they existed, were required to allow storing a pointer in an integer type and converting it back again.

P3248R0 made this type mandatory for C++23.

On most mainstream systems, a `uintptr_t` is an integer type that is the same size as `void*` and contains a linear address in the target's address space.

In CHERI C/C++, this type is implemented by a compiler built-in type (`__intcap_t`), which implements the full set of integer arithmetic operations by operating on the address.

For example, consider this function that clears low bits in a pointer to ensure that it is correctly aligned (various variations of this exist in widely deployed real-world codebases):

```
int *force_align(int *num) {
    uintptr_t address = reinterpret_cast<uintptr_t>(num);
    return reinterpret_cast<int*>(address & (alignof(int))-1);
}
```

CHERI Clang first generates LLVM IR like this (for a 64-bit CHERI RISC-V target):

```
%0 = tail call i64 @llvm.cheri.cap.address.get.i64(ptr addrspace(200) %num)
%and = and i64 %0, 3
%1 = tail call ptr addrspace(200) @llvm.cheri.cap.address.set.i64(ptr addrspace(
ret ptr addrspace(200) %1
```

Note that the `and` operation is bracketed by a get-address and set-address intrinsic.

Any operation on the address is valid, but if the resulting address is out of the original bounds then any attempt to use the pointer will fail.

When targeting 64-bit (pre-standard) CHERI RISC-V, this will be two instructions:

```
andi    a1, a0, 3
csetaddr ca0, ca0, a1
cret
```

The get address is implicit because register `a0` is the address portion of register `ca0`.

The set address is an explicit instruction that takes an existing capability

Various CHERI projects have run large C and C++ codebases totalling over 100 MLoC, including the FreeBSD kernel and userland, the KDE Wayland compositor (including Arm Mali GPU drivers) and a set of KDE applications, and the Chromium web browser, with this interpretation of `[u]intptr_t` semantics.

Uses of pointers as integers

There are two distinct classes of operation that require accessing a pointer as a numerical value.

They are differentiated by whether they will ever convert back to a pointer.

This is important for optimisation.

Typically, a pointer that is converted to an integer is treated as having escaped.

There may be other operations later that materialise a pointer to the same object that the compiler can't reason about.

In C++ today, there is no way of expressing this distinction.

The use of `std::hash<T*>`, for example, hashes a pointer using its address.

This involves a conversion from the `T*` to some integer type but should not be treated as an escaping operation because there is never conversion back from that integer to the original pointer.

On mainstream hardware, this leads to missed optimisation opportunities.

In CHERI C/C++, this distinction is important because the storage size of a provenance-free integer that contains just the address is less than the storage size of a pointer.

To avoid this, CHERI C/C++ introduced a new type: `ptraddr_t`.

Converting from `ptraddr_t` to `T*`, on a CHERI platform, is *guaranteed* to give a pointer that cannot be dereferenced.

C++ already permits most pointer-as-integer operations without round trips through `intptr_t`

C++ permits converting any pointer to a `char*` and then performing arithmetic and converting back, as long as the final cast type is valid for the object at its new address.

If you have some a-priori knowledge of the address, you can always rewrite arithmetic on a pointer as an add operation on the pointer as a `char*` that yields the same result.

For example, consider the following operation that rounds down:

```
uintptr_t address = reinterpret_cast<uintptr_t>(x);
x = reinterpret_cast<decltype(x)>(address & 7);
```

You can do the same operation with a non-escaping variant, if you assume the existence of a `get_address` function that takes a pointer and returns its address some integer type that can hold any address, which we will call `ptraddr_t`:

```
ptraddr_t currentAddress = get_address(x);
ptraddr_t roundedAddress = currentAddress & 7;
// Volatile / const ignored for clarity
x = reinterpret_cast<decltype(x)>(reinterpret_cast<char*>(x) + (roundedAddress - c
```

As a principle, this paper aims to define more ergonomic, optimiser-friend, and CHERI-friendly behaviours for operations that can already be expressed with arithmetic on a `char*`, and address some of the provenance-related soundness issues with the current implementations.

Core Proposal

The proposal introduces a new type, `ptraddr_t` that can hold an address, defines semantics that must be supported for `ptraddr_t` and `intptr_t` that reflect existing usage, and defines new APIs inspired by Rust's strict provenance mode that provide a cleaner approach that could be a core building block for a future strict provenance profile.

The `ptraddr_t` type

The `ptraddr_t` type is a *provenance-free* integer type that can hold an address.

Any pointer can be converted to this type via the APIs introduced below.

The integer value must have the following properties:

- Two `ptraddr_t` values from different objects must be distinct.
- Two in-bounds pointers to different offsets within of the same object must yield different `ptraddr_t` values.
- If two pointers are converted to `char*` and one subtracted from the other, the result must be the same as the result of first converting both pointer to `ptraddr_t` and subtracting.

Specifically, $\text{reinterpret_cast}\langle\text{char}*\rangle(x) - \text{reinterpret_cast}\langle\text{char}*\rangle(y) == \text{get_address}(x) - \text{get_address}(y)$ for all values of `x` and `y` for which the first subtraction is defined.

This also implies that `offsetof` calculations give the same difference.

- All casts from `ptraddr_t` to a pointer type, including those via an intermediate type such as `intptr_t` are undefined and (as a QoI issue) implementations are encouraged to diagnose these operations where possible.

Systems with segmented architectures may require a large `ptraddr_t` to include both the segment descriptor and the offset.

Why not `size_t` ?

In early CHERI C work, `size_t` was used in the places where `ptraddr_t` is now used.

This introduced compatibility problems with some C code written for embedded platforms.

`size_t` is required to hold the largest size for any *individual* object.

We were informed that systems with separate ROM and RAM are limited to 64 KiB banks of memory and so use a 16-bit `size_t`, a 32-bit `void*` and `char*`, but 16-bit pointers when explicitly qualified with an address space.

Similarly, a proposed ABI for RV128 suggested using 64-bit `size_t` and 128-bit pointers, limiting individual allocations to 16 EiB.

Subtraction on `ptraddr_t`

There is discussion over whether the result of subtraction of two `ptraddr_t` values should be `ptraddr_t` or `ptrdiff_t`.

There are arguments in favour of either interpretation.

The benefit of treating the result as `ptrdiff_t` is that this matches the semantics of subtraction on two pointers, and the value for subsequent addition to pointers.

Nevertheless, there are several reasons why `ptraddr_t` may be a more desirable choice:

- No other integer type in C or C++ has the property that `decltype(x-x)` is not `decltype(x)` and so this would be new semantics to both implement in compilers and explain to programmers.
- On most implementations, `ptraddr_t` would be a simple typedef for some other integer type, and so this would require special handling.
- C and C++ make it undefined behaviour to subtract two pointers to different objects. This is well-defined behaviour for two addresses created by converting pointers to `ptraddr_t` and these may be anywhere in the address space, including one in the kernel and one in userspace for a kernel, which could lead to overflow if the result had to be a signed value, triggering undefined behaviour.
- Unsigned overflow is well-defined behaviour and so if the result of a sequence of addition and subtraction on `ptraddr_t` wraps the address space, reapplying it will always work.
- Integer promotion rules become complex if the types do not match.

If `a`, `b`, and `c` are all `ptraddr_t`, `a - b - c` would first evaluate `a - b` as `ptrdiff_t` and then *may* result in integer promotion for either depending on the underlying types.

In contrast, if `a - b` is a `ptraddr_t` then the type is consistent throughout and there is no risk of undefined behaviour as a result of integer promotion.

- Addition of two `ptraddr_t` values should result in a `ptraddr_t` (there is no reason that adding two unsigned values should result in an unsigned value), and so addition and subtraction would have different rules.

Similarly, multiplication, bit shifting, and so on (all valid operations for, for example, hashing addresses) would not result in the result being `ptrdiff_t` and so subtraction would be unlike every other arithmetic operation.

Mandatory operations on `intptr_t`

NOTE: For the remainder of this document, `intptr_t` is used as a shorthand for `intptr_t` or `uintptr_t`.

C++23 makes the `intptr_t` type mandatory, but leaves most operations on it implementation defined.

This paper recommends standardising existing usage by defining the set of operations on `intptr_t` that can be expressed in terms of `char*` operations and address access.

Specifically:

Arithmetic on a pair of `uintptr_t` values where the first is constructed from a pointer and the second constructed from an integer value should be implemented to preserve the following equivalence:

```
uintptr_t fromObject = reinterpret_cast<uintptr_t>(someObject);
uintptr_t fromInteger = someIntegerValue;
T *viaIntPtr = reinterpret_cast<T*>(fromObject {op} fromInteger);

ptraddr_t originalAddress = get_address(someObject);
ptraddr_t modifiedAddress = originalAddress {o} someIntegerValue;
T *viaChar = reinterpret_cast<T*>(reinterpret_cast<char*>(someObject) + (modified/

assert(viaChar == viaIntPtr);
```

If the address arithmetic that calculates `viaChar` is undefined behaviour then it is undefined unless noted below.

Arithmetic on two `intptr_t` values where neither is derived from a pointer must have the same semantics as if it were performed on two `ptraddr_t` values.

Note; however, that the size of `intptr_t` may be larger than `ptraddr_t`, with some bits that are unused for simple address or integer arithmetic.

For example, on a 64-bit CHERI RISC-V system, `intptr_t` and `void*` are 16 bytes, whereas `ptraddr_t` is 8, yet the range of arithmetic for `intptr_t` is the same as that for `int64_t`.

These two requirements do not permit anything that cannot already be done in the C++ abstract machine via other mechanisms and that are assumed to work in a large body of existing C++ code.

They trivial to implement on mainstream systems and feasible to implement on CHERI systems as well as systems with segmented memory.

The following options for implementation-defined behaviour that should be declared via a feature-test macro:

- Operations via `intptr_t` that combine two values derived from pointers (such as an XOR linked list) will result in a valid pointer.
- Operations on `intptr_t` are symmetric.
- Operations via `intptr_t` that materialise a pointer to an unrelated but valid (in bounds, live) object will give a valid pointer.

It is expected that all of these will be set on most existing non-CHERI systems.

As always, implementations are free to explicitly define any other undefined behaviour.

Limitations with `intptr_t`

Operations on `intptr_t` are defined as operating on a pair of `intptr_t` values.

Consider the following:

```
uintptr_t x = reinterpret_cast<uintptr_t>(someObject);
uintptr_t y = 42;
T *object = reinterpret_cast<T*>(x + y);
```

In the `x + y` expression, where does the provenance come from?

The CHERI C/C++ compiler currently issues a warning in this case.

By convention, it assumes that the first operand is the might-be-a-pointer operation and so this operation works, but rewriting it as `y + x` would result in a pointer that cannot be dereferenced (it would be the untagged CHERI capability with the value 42, added to the address of `someObject`).

New strict-provenance APIs

The proposed new APIs address the limitations of using `intptr_t` for both kinds of pointer-as-address operation.

The following is a synopsis of the proposed APIs:

XXX David: I think it would be good to split this into two parts.

The first wants to introduce the `get_address` and `with_address` functions, the with the atomic bits.


```

1  //////////////////////////////////////
2  // <stdint>
3  using ptraddr_t = integer suited to hold a pointer address;
4  // - See expr.reinterpret.cast#4, expr.reinterpret.cast#5:
5  //   May not suffice to reinterpret_cast pointer to integer and back.
6
7  //////////////////////////////////////
8  // <memory>
9
10 // Obtains the address of a pointer
11 ptraddr_t get_address(void* p) noexcept;
12 // Note: for use cases that only require a pointer address, it may
13 // provide better performance or wider platform support, since
14 // implementations are not required to expose the provenance of the
15 // pointer.
16
17 // Create pointer with `address` to same allocation as `p`:
18 template <typename T>
19 T* with_address(T* p, ptraddr_t address);
20
21 // (utility): Maps the address of a pointer
22 template <typename T, typename UF>
23 constexpr T* transform_address(T* p, UF f)
24     requires(invokable<UF, ptraddr_t>
25              && same_as<invokable_result_t<UF, ptraddr_t>, ptraddr_t>)
26 {
27     return with_address(p, f(get_address(p)));
28 }
29
30 //////////////////////////////////////
31 // <atomic>
32
33 // See below:
34 template <typename T>
35 class atomic_ref<T*> {
36     constexpr bool compare_exchange_weak(
37         T*&, T*,
38         T*& provenance_target,
39         memory_order, memory_order
40     ) const noexcept;
41     constexpr bool compare_exchange_strong(
42         T*&, T*,
43         T*& provenance_target,
44         memory_order, memory_order
45     ) const noexcept;
46     constexpr bool compare_exchange_weak(
47         T*&, T*
48         T*& provenance_target,
49         memory_order = memory_order::seq_cst
50     ) const noexcept;

```

```

51     constexpr bool compare_exchange_strong(
52         T*&, T*,
53         T*& provenance_target,
54         memory_order = memory_order::seq_cst
55     ) const noexcept;
56 };
57
58 // analogous APIs for atomic<T*> and free functions

```

The new compare-exchange APIs atomically perform the following steps to avoid undefined behavior due to ABA:

```

1  bool compare_exchange_<weak|strong>(
2      this, T*& expected_ptr, T* replacement_ptr, T*& provenance_target_provenance
3      memory_order success, memory_order failure
4  ) {
5      // atomically performs the following:
6
7      // Reads value from memory:
8      auto current = atomic_load_explicit(*this, failure);
9      // Obtain its address:
10     auto current_addr = get_address(current);
11     auto expected_addr = get_address(expected_ptr);
12     if (current_addr != old_addr) {
13         // failure:
14         return false;
15     }
16     // success:
17
18     // Update provenance of target_provenance with the provenance
19     // of current:
20     target_provenance = with_address(current, get_address(target_provenance));
21     store(this, replacement_ptr, success);
22     return true;
23 }

```

Operations on pointers set with `with_address` are defined such that the following two paths are equivalent:

```

ptraddr_t original = get_address(x);
ptraddr_t new = some_permutation_on(original);

// The following two should be equivalent:
T *x_with_address = with_address(x, new);
// Or this
char *x_as_char = reinterpret_cast<char*>(x);
x_as_char += (new - original);
T *x_via_char = reinterpret_cast<decltype(*x)>(x_as_char);

assert(x_with_address == x_via_char);

```

Unlike the `intptr_t` version, it is *always* undefined behaviour to dereference `x_with_address` if the result is not in the bounds of `x` and the type of `T` is not compatible with the underlying storage value.

Examples

Concurrent queue

This example adapted from Jonas Böttiger and Ralf Jung provenance-related Rust `RWLock` bug (<https://github.com/rust-lang/rust/issues/121950>). Alice Rhyl showed a similar example provenance-related concurrent-stack bug (<https://github.com/rust-lang/unsafe-code-guidelines/issues/480>), which is similar to the LIFO stack described in P2414R3 (pointer zap) (<https://wg21.link/p2414r3>) and in P2434R3 (non-deterministic pointer provenance) (<https://wg21.link/p2434r3>).

```

1  struct Node {
2      Node* next;
3  };
4
5  struct Queue {
6      atomic<tagged_ptr<Node*>> head;
7  };
8
9  void enqueue(Queue& q, Node* new_head_node) {
10     tagged_ptr<Node*> old_head = q.head.load();
11     do {
12         new_head_node->next = old_head.get();           // pointer masking
13         tagged_ptr<Node*> new_head{new_head_node}.set(); // bittagging
14     } while(!q.head.compare_exchange_strong(old_head, new_head));
15 }

```

This example exhibits undefined behavior due to ABA:

1. Thread 0 `old_head` load reads pointer to node with addr `A` and provenance `p0`.
2. Thread 1 deallocates node `A`, allocates a new node that gets the same address (but different provenance `p1`), and enqueues it.
3. Thread 0 compare-exchange succeeds due to `A` comparing equal, which writes `new_head` to memory, which contains an internal pointer with address `A` and provenance `p0` (deallocated node).
4. Thread 2 attempts to traverse the list and exhibits undefined behavior of the form use-after-free due to `head->next` having provenance `p0` to a deallocated object, instead of `p1`.

The new `compare_exchange_strong` makes the algorithm well-defined:

```
1 void enqueue(Queue& q, Node* new_head_node) {
2     tagged_ptr<Node*> old_head = q.head.load();
3     do {
4         new_head_node->next = old_head.get();           // pointer masking
5         tagged_ptr<Node*> new_head{new_head_node}.set(); // bittagging
6     } while(!q.head.compare_exchange_strong(old_head, new_head,
7         // uses new CAS API to update the provenance of next
8         // before head is updated in memory:
9         &new_head_node->next));
10 }
```

and step 3 above changes as follows:

3. Thread 0 compare-exchange succeeds due to `A` comparing equal, which writes `new_head` to memory, which contains an internal pointer with address `A` and provenance ~~`p0` (deallocated node)~~ `p1` (new node).

The new `compare_exchange_strong` API updates the new head's `next` pointer to point to the new allocation as part of atomically updating the head, avoiding UB when traversing the list.

C++ programs can avoid UB in these examples without the proposed explicit provenance APIs, e.g., by performing an extra load inside the CAS loop, and then exposing the provenance of the loaded pointer, such that an angelic provenance can be picked that refers to the new allocation, e.g., via an explicit `uintptr_t` round-trip; see P2414R3 (pointer zap) (<https://wg21.link/p2414r3>) and P2434R3 (non-deterministic pointer provenance)

(<https://wg21.link/p2434r3>). The explicit provenance APIs avoid exposing the pointer provenance and avoid requiring the programmer to understand the angelic provenance model. It also saves tools (interpreters, sanitizers) from having to scan all memory to find candidate provenances.

Implementation Experience

There is prior implementation experience on the Rust side, see Rust Tracking Issue for Strict Provenance (<https://github.com/rust-lang/rust/issues/95228>).

The CHERI version of LLVM/Clang has had working C++ support since 2017 and has been used to compile over 100MLoC with this interpretation of `intptr_t` semantics.

The proposed strict-provenance APIs map 1:1 with the LLVM intrinsics that are used to implement this behaviour and so implementation is trivial.

In CHERI C++ on clang, the implementation is simply:

```
ptraddr_t get_address(void* pointer)
{
    return __builtin_cheri_address_get(pointer);
}

template<typename T>
T *with_address(T *pointer, ptraddr_t address)
{
    // Note: No casts are necessary here because the return type of
    // __builtin_cheri_address_set is the same as the first argument.
    return __builtin_cheri_address_set(pointer, address);
}
```

Optional extension

A C++ *strict provenance* profile (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3589r2.pdf>) that makes reinterpret casts from pointers to integers and vice-versa ill-formed, enabling applications to be incrementally updated to use these new APIs for explicitly handling provenance instead.

C compatibility

Currently, C does not mandate `intptr_t` and so has no portable way of doing any of the operations described in this paper.

In practice, almost all C implementations do include `intptr_t` and most C code relies on it behaving in this way.

We would propose our rules for `intptr_t` being added to C, because any C implementation that allows casting to and from `char*` and performing arithmetic (i.e. any that compiles with the standard) should be able to implement these semantics.

We would also propose the `get_address` and `with_address` functions be added as functions or function-like macros in C.

A C implementation for existing architectures might look like this:

```
#define get_address(pointer) ((ptraddr_t)(pointer))
#define with_address(pointer, address) \
    ((typeof((pointer)))(((char*)(pointer))+address))
```

Or for CHERI systems like this:

```
#define get_address(pointer) \
    __builtin_cheri_address_get((pointer))
#define with_address(pointer, address) \
    __builtin_cheri_address_set(pointer, address)
```

Note on naming

The `get_address` and `with_address` identifiers are likely to conflict with existing code.

In C++, placing them in the `std` namespace would eliminate this conflict.

In C, they may need to be named `_GetAddress` / `_WithAddress` or similar.

The names of identifiers in this paper should not be assumed to be final.

Appendix: XOR linked list

XOR linked lists are not compatible with a strict single-provenance model, *unless* the provenance can be reconstructed from elsewhere.

The following example shows an XOR linked list implementation that works on CHERI systems, which have a hardware-enforced single-provenance model.

This relies on allocating all list nodes from a single arena, which allows reconstructing

provenance.

It could be generalised to allocating from a small pool of arenas, amortising the cost of lookup and storage of the full pointers.

```

/**
 * A minimal XOR linked list implementation.
 *
 * This demonstrates that an XOR linked list is possible with strict provenance
 * *assuming* a mechanism for recovering provenance.
 *
 * This is *not* intended as an example of how to create an XOR linked list, it
 * contains a large number of poor practices. It does; however, run on CHERI
 * C++, with a hardware-enforced single provenance model.
 *
 * The template parameters are the type of the list and the size of the arena
 * to allocate this list from (a real implementation would also have a tag
 * type to disambiguate the arenas).
 */
template<typename T, size_t ArenaSize=30>
class XORList
{
    /**
     * The arena from which this list is allocated.
     */
    static char *arena()
    {
        static char *listArena = new char[ArenaSize*sizeof(XORList)];
        return listArena;
    }

    /**
     * Bump allocate space for a new list node from the arena. Returns null if
     * the arena is exhausted.
     */
    static char *allocate()
    {
        static int i=0;
        if (i>=ArenaSize)
        {
            return nullptr;
        }
        return &arena()[i * sizeof(XORList)];
    }

    /**
     * A single address-sized word containing the next and previous pointers
     * xor'd together. Either is therefore recoverable if the other is known.
     */
    ptraddr_t nextXorPrevious = 0;

    /**
     * Private constructor, these must be allocated from the pool. For simpli
     */

```

```

XORList(T &value) : value(value) {}
public:
/**
 * Head of the list.
 */
inline static XORList *head;

/**
 * Tail of the list.
 */
inline static XORList *tail;

/**
 * The body of the list node.
 */
T value;

/**
 * Create a new list node and add it to the end of the list.
 */
static XORList *emplace_back(T &value)
{
    char *nextSpace = allocate();
    if (!nextSpace)
    {
        return nullptr;
    }
    XORList *next = new (nextSpace) XORList (value);
    if (head == nullptr)
    {
        head = next;
        tail = next;
        next->nextXorPrevious = 0;
    }
    else
    {
        tail->nextXorPrevious ^= get_address(next);
        next->nextXorPrevious = get_address(tail);
        tail = next;
    }
    return next;
}

/**
 * Returns the next pointer, given a previous pointer.
 *
 * Alternatively, returns a previous pointer given a next pointer.
 */
XORList *next(XORList *previous)

```

```

        {
            auto previousAddress = get_address(previous);
            auto nextAddress = nextXorPrevious ^ previousAddress;
            return with_address(reinterpret_cast<XORList*>(arena()), nextAddress);
        }
};

/**
 * Create a list of the numbers 0-19.
 */
void allocate_list()
{
    for (int i=0 ; i<20 ; i++)
    {
        XORList<int>::emplace_back(i);
    }
}

/**
 * Traverse the list in either direction. The `first` argument must be the
 * head or tail of the list. Prints the values of each list node.
 */
void traverse_list(XORList<int> *first)
{
    XORList<int> *previous = nullptr;
    XORList<int> *current = first;
    while (current != nullptr)
    {
        printf("%d\n", current->value);
        XORList<int> *next = current->next(previous);
        previous = current;
        current = next;
    }
}

void list_example()
{
    allocate_list();
    traverse_list(XORList<int>::head);
    traverse_list(XORList<int>::tail);
}

```

References

1. CHERI C/C++ Programming Guide (<https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-947.pdf>).

2. *Zaliva et al.*, Formal Mechanised Semantics of CHERI C: Capabilities, Undefined Behaviour, and Provenance (<https://www.cl.cam.ac.uk/~pes20/asplos24spring-paper110.pdf>), ASPLOS '24.
3. Aria Desires, Rust's Unsafe Pointer Types Need An Overhaul (<https://faultlore.com/blah/fix-rust-pointers>), '22.
4. Ralf Jung, Pointers Are Complicated III, or: Pointer-integer casts exposed (<https://www.ralfj.de/blog/2022/04/11/provenance-exposed.html>), '22.
5. Rust Tracking Issue for Strict Provenance (<https://github.com/rust-lang/rust/issues/95228>).